

编辑器与生命周期说明

本页面介绍了一些 Unity 特有的行为，这些行为通常解释了为什么 MIDI 在编辑器中与在实际构建中的表现会有所不同。

管理器生命周期与 DontDestroyOnLoad

在播放模式下，第一次访问 `MidiManager.Instance` 时，会创建一个带有 `DontDestroyOnLoad` 属性的 `GameObject`。

实践影响：

- 您通常在每次播放会话中仅初始化一次。
- 请勿在 Unity 未处于播放模式时通过编辑器工具调用管理器。

Unity 编辑器播放模式转换

本项目包含了编辑器挂钩 (Editor hooks)，以便传输协议在进入/退出播放模式时能干净地启动/停止。

如果您遇到以下情况：

- 停止播放模式后仍有 MIDI 信号传入，或者
- 设备未能正常关闭，

请确保：

- 您在关机时终止了 MIDI (`TerminateMidi` / `TerminateMidi2`)；
- 您的脚本没有在不同播放会话之间保留静态引用。

主线程分发

某些传输协议（例如网络传输）在后台线程接收消息。传入的事件会以主线程安全的方式转发给 Unity。

实践影响：

- 您的事件处理器可以假定回调发生在 Unity 的主线程上。
- 仍应避免在回调中进行繁重的处理；如有需要，请将工作加入队列。

EventSystem 注意事项

如果您的场景中已经包含 Unity `EventSystem`，请避免创建重复的实例。

如果您遇到重复冲突的警告/错误，请相应地调整初始化代码（参见 [入门指南](#)）。

MIDI 监视器 (编辑器工具)

Unity 编辑器包含一个 **MIDI 监视器**，可在播放模式下实时显示 MIDI 输入/输出。

```
Window > MIDI > Monitor
```

行为

- 播放模式开始时会自动创建一个 `[MIDI Monitor Proxy]`，并将其作为事件处理器注册到 `MidiManager`。
- 播放模式结束时，该 Proxy 会被自动销毁。
- 发出的消息通过 `MidiManager` 的编辑器专用挂钩（`#if UNITY_EDITOR`）捕获。

注意事项

- 监视器仅在播放模式下可用。在编辑模式下不会显示消息。
- Proxy 的创建独立于 `MidiManager.InitializeMidi()`。即使在 MIDI 初始化之前，Proxy 也会被注册，但要接收消息仍需像往常一样调用 `InitializeMidi()`。
- 日志缓冲区在停止播放后仍会保留在窗口中。您可以在下次播放时通过 **Clear** 将其清除。

详情请参见 [编辑器工具 \(MIDI 监视器\)](#)。